Facultad de Ingeniería en Sistemas

Construcción y Evolución del Software

Integrantes:

Chauca Rojas Aaron Ernesto

Sarmiento Caillagua Néstor David

Grupo # 5

GR2SW

Tarea 1: Desarmando Gigantes - Ingeniería Inversa en la Práctica

Fase 1: Contextualización

- **Ingeniería inversa**

  La ingeniería inversa, aplicada al desarrollo de software, consiste en analizar un sistema ya construido para comprender cómo funciona internamente, especialmente cuando no se dispone de su diseño original o documentación. En otras palabras, implica "echarse para atrás" en el ciclo de vida del software, pasando de un producto terminado hacia sus componentes, lógica y estructura. Este proceso permite reconstruir información sobre el sistema, identificar cómo se comunican sus partes y entender las decisiones de desarrollo que se tomaron, lo cual resulta útil cuando se necesita modificar, mejorar o integrar ese software con otros sistemas.

- **Mantenimiento de software**

  El mantenimiento de software se refiere al conjunto de actividades que se realizan después de que un sistema ha sido implementado, con el fin de corregir errores, adaptarlo a nuevos entornos o mejorar su funcionamiento. Existen diferentes tipos de mantenimiento, entre ellos el correctivo, que soluciona fallos; el adaptativo, que permite que el software funcione en nuevos contextos tecnológicos; el perfectivo, que mejora el rendimiento o la experiencia del usuario; y el preventivo, que busca evitar futuros problemas. En este contexto, la ingeniería inversa es fundamental, ya que muchas veces el software no cuenta con documentación actualizada, por lo que es necesario analizarlo para entender su estructura antes de poder realizar cambios de manera segura.

- **Evolución del software**

  La evolución del software hace referencia al proceso continuo mediante el cual un sistema se modifica y mejora a lo largo del tiempo para mantenerse útil, eficiente y acorde a las necesidades actuales. La ingeniería inversa juega un papel clave en este proceso, ya que permite comprender sistemas antiguos o complejos que, de otro modo, serían difíciles de actualizar. Gracias a este análisis, es posible identificar componentes obsoletos, mejorar la arquitectura y adaptar el software a nuevas tecnologías, evitando que el sistema quede desactualizado o pierda funcionalidad frente a las demandas del entorno.

Fase 2. Antes, ahora y mañana

Antes

En los años 90, la ingeniería inversa de software estaba muy centrada en programas compilados para escritorio y sistemas legacy. Las técnicas principales eran el desensamblado, la inspección del binario en lenguaje ensamblador, el uso de depuradores y la observación manual del comportamiento del programa. Herramientas históricas de esa época y de esa transición incluyen desensambladores tempranos como IDA, cuyo desarrollo comenzó entre 1990 y 1991, y depuradores de bajo nivel que permitían examinar la ejecución instrucción por instrucción. El trabajo era muy manual, dependía mucho del conocimiento del procesador, de formatos binarios y del sistema operativo.

Ahora

Con la llegada de aplicaciones móviles, servicios web, cloud computing y microservicios, la ingeniería inversa evolucionó. Ya no basta con mirar un ejecutable: ahora también hay que estudiar tráfico de red, APIs, tokens, comportamientos cliente-servidor, almacenamiento local y mecanismos de autenticación. Herramientas modernas como Ghidra integran desensamblado, descompilación y análisis gráfico; JADX permite convertir APK/Dex en código Java legible; Frida habilita instrumentación dinámica de procesos “caja negra”; y proxies como mitmproxy o Burp Proxy permiten interceptar, inspeccionar y modificar tráfico HTTP/HTTPS y WebSockets durante pruebas autorizadas. Este cambio refleja que el software actual es mucho más distribuido y ofuscado que el de décadas anteriores.

Novedades e IA

Hoy la novedad más importante es el uso de Inteligencia Artificial para asistir la ingeniería inversa. Los modelos actuales pueden ayudar a interpretar pseudocódigo, renombrar funciones, inferir intención semántica, clasificar malware y apoyar el análisis de vulnerabilidades a partir de binarios descompilados. La investigación reciente muestra que los enfoques basados en LLM están mejorando la comprensión humana del código descompilado, aunque todavía pueden fallar en fidelidad funcional frente a herramientas clásicas. En otras palabras, la IA todavía no reemplaza al analista, pero sí acelera la documentación, la hipótesis arquitectónica y la priorización de componentes críticos.

Fase 3: El caso de estudio – “Hacking hipotético de Cashew”

Selección del objetivo

Para el desarrollo de este caso de estudio se seleccionó la aplicación Cashew, una plataforma utilizada para la gestión de finanzas personales. Dentro de sus múltiples funcionalidades, se eligió analizar el sistema de sincronización y respaldo de datos entre dispositivos, ya que implica la interacción de varios componentes del sistema, como almacenamiento local, servicios en la nube y comunicación con APIs. El objetivo de este análisis es comprender cómo opera esta funcionalidad sin tener acceso al código fuente original, aplicando un enfoque de ingeniería inversa basado en observación externa.

Técnicas de ingeniería inversa

Análisis de caja negra

En primer lugar, se aplicaría un enfoque de análisis de caja negra, el cual consiste en interactuar con la aplicación como lo haría un usuario común, observando cómo responde ante acciones como la creación, modificación o eliminación de transacciones, así como el uso sin conexión y la posterior sincronización. A través de este proceso es posible identificar patrones de comportamiento, tiempos de respuesta y reglas implícitas del sistema, lo que permite inferir cómo se gestionan los datos internamente sin necesidad de conocer su implementación.

Análisis de tráfico de red

En segundo lugar, se emplearía el análisis de tráfico de red, utilizando herramientas que permiten inspeccionar las solicitudes y respuestas intercambiadas entre la aplicación y sus servidores en un entorno controlado. Esta técnica facilita la comprensión de cómo se transmiten los datos, qué tipo de estructuras se utilizan y cómo se gestionan procesos como la autenticación y la sincronización, permitiendo deducir la existencia de APIs y servicios backend.

Ingeniería inversa de la aplicación móvil

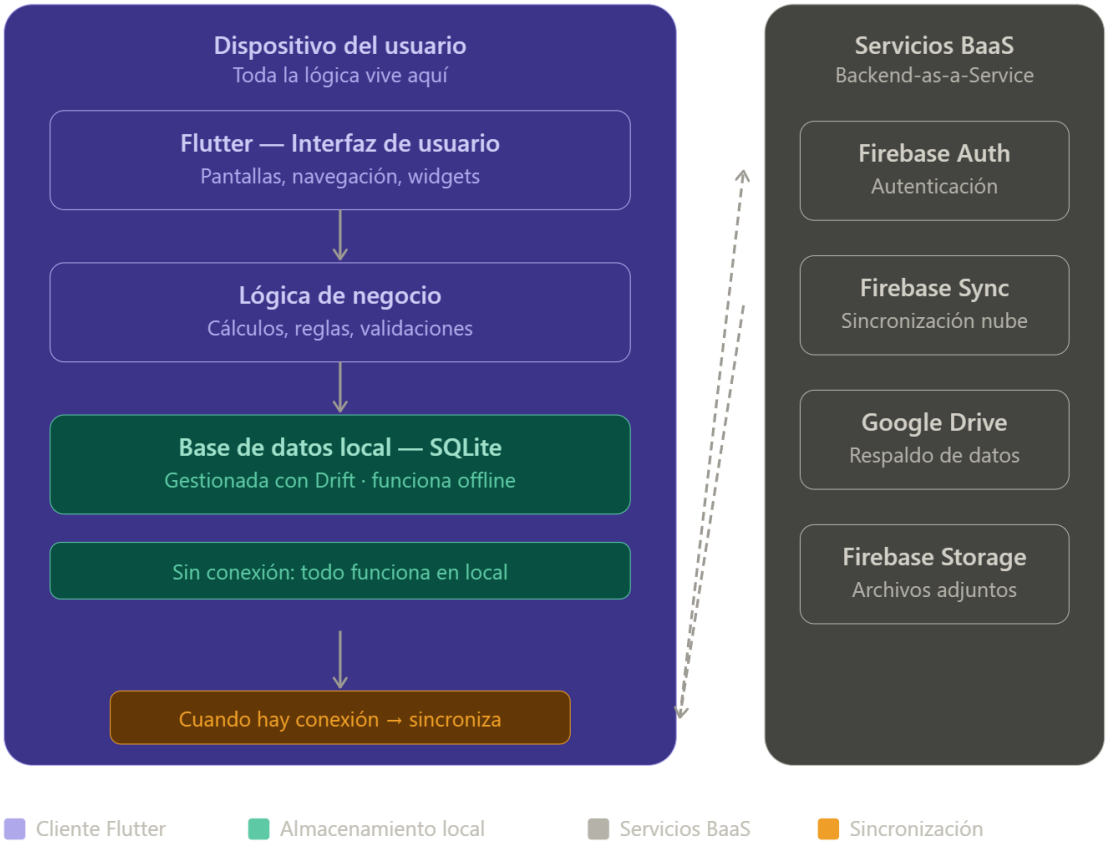
Finalmente, se consideraría la ingeniería inversa de la aplicación móvil mediante la descompilación del archivo APK en el caso de Android, lo que permitiría obtener una representación aproximada del código y analizar la estructura interna de la aplicación. Este proceso ayudaría a identificar módulos relacionados con la sincronización, el almacenamiento de datos y los mecanismos de autenticación, complementando la información obtenida a través de las otras técnicas.

Hipótesis de arquitectura

Cashew utiliza una arquitectura de cliente pesado (thick client) con enfoque offline-first. En este modelo, el frontend desarrollado en Flutter no solo se encarga de la interfaz de usuario, sino también de gran parte de la lógica de negocio.

En lugar de un backend tradicional con API REST propia, la aplicación se apoya en servicios de *Backend-as-a-Service* (BaaS) como Firebase para funcionalidades como autenticación, sincronización y almacenamiento en la nube. La gestión de datos ocurre principalmente de forma local a través de una base de datos SQLite (gestionada con Drift), lo que permite usar la app sin conexión a internet y sincronizar la información con servicios externos, como Firebase o Google Drive, cuando hay conectividad disponible.

Esta arquitectura simplifica considerablemente la infraestructura del servidor, pero centraliza la responsabilidad en el dispositivo del usuario, priorizando el rendimiento, la disponibilidad sin conexión y la simplicidad operativa.



Conclusión

La ingeniería inversa permite comprender cómo funciona un sistema incluso sin acceso a su código fuente, lo que resulta fundamental para su mantenimiento y evolución. En este trabajo, se aplicaron técnicas como el análisis de caja negra, el análisis de tráfico de red y la descompilación para estudiar la aplicación Cashew, logrando inferir su funcionamiento y arquitectura. Esto demuestra que, en el contexto actual, la ingeniería inversa sigue siendo una herramienta clave para analizar y entender aplicaciones modernas basadas en servicios en la nube y almacenamiento local.

Imagen:

Prompt: Ilustración digital 3D estilo isométrico futurista de la aplicación financiera "Cashew", representada como un gran teléfono inteligente brillante. El dispositivo está siendo analizado por ingenieros digitales. Uno observa cuidadosamente la pantalla y el comportamiento de la app (representando análisis de comportamiento del usuario), otro escanea flujos de datos que salen del dispositivo hacia la nube con un rayo de luz (representando análisis de comunicación de red), y otro está abriendo el teléfono en capas para ver sus componentes internos (representando análisis interno del sistema). Dentro del dispositivo se revelan componentes como una base de datos local, servicios en la nube tipo Firebase, autenticación y respaldo de datos. Líneas de datos luminosas conectan el teléfono con una nube flotante. El logo de Cashew aparece en la interfaz. Estilo cyberpunk, iluminación neón azul y violeta, muy detallado, alta calidad.